

1 Commitment Schemes

A commitment scheme is a cryptographic primitive that allows a party to commit to a value without revealing the value itself. At a later stage, the party can reveal the value and prove that it is the same value to which it committed earlier.

A commitment therefore provides two basic properties: *hiding* and *binding*.

1.1 Hiding

The hiding property ensures that the commitment does not reveal the committed value. Given only the commitment, the receiver should not be able to determine the value that was committed.

1.2 Binding

The binding property ensures that, after producing a commitment, the sender cannot later open the same commitment to two different values.

Thus, the two properties can be summarized as:

- **Hiding:** The receiver cannot learn the committed value from the commitment.
- **Binding:** The sender cannot change the committed value after committing.

The commitment process is often viewed in two phases. In the *commit phase*, the sender produces a commitment to a value m , usually using some randomness r . In the *opening phase*, the sender reveals m and r , allowing the receiver to verify the commitment.

2 Example: Committing to a Future Grade

Suppose a teacher wants to commit today to whether a student's final grade will be at least B , without revealing the answer.

Let b denote the teacher's decision, where $b = 1$ if the final grade is at least B and $b = 0$ otherwise.

The teacher commits to b before the final grade is known. Later, after the result is available, the teacher reveals b and provides the information necessary to verify that the same value was committed earlier.

The important requirement is that the teacher should not be able to change the value after making the commitment.

This example illustrates the two properties clearly. The value must remain hidden before the appropriate time, while the commitment must prevent the teacher from changing the value later.

3 Hash-Based Commitment

A simple commitment scheme can be constructed using a cryptographic hash function H .

The sender chooses a random value $r \leftarrow \{0, 1\}^\lambda$ and computes

$$c = H(m \| r).$$

Only c is sent to the receiver.

The random value r acts as a hiding value. Even if the message m comes from a small set, the receiver should not be able to determine which message was committed by observing the commitment alone.

To open the commitment, the sender reveals (m, r) . The receiver computes $H(m \| r)$ and checks whether it equals the original commitment c .

If the values match, the receiver accepts the opening.

3.1 Why is the Commitment Binding?

Suppose the sender initially committed using (m, r) but later attempts to open the same commitment using a different message $m' \neq m$ and some value r' .

For the alternative opening to be accepted, it must satisfy

$$H(m\|r) = H(m'\|r').$$

Finding such an alternative opening should be computationally infeasible for a suitable cryptographic hash function.

Thus, the binding property follows from the collision resistance of the hash function.

3.2 Why is the Commitment Hiding?

The receiver sees only $c = H(m\|r)$ and does not know the random value r . For a suitable cryptographic hash function and sufficiently large randomness, the receiver should not be able to determine m from c .

The random value is particularly important when the message space is small. Without randomness, an attacker could simply compute the hash of every possible message and compare it with the commitment.

4 Committing to Different Values

The commitment need not be restricted to a single bit.

For example, suppose the student's final mark is $m \in [0, 100]$. The teacher can directly commit to the mark by computing

$$c = H(m\|r).$$

The teacher later reveals (m, r) , and the student verifies the commitment by recomputing the hash.

Thus, the commitment mechanism can be used for a bit, an integer, a message, or other types of data.

5 Commitment versus Zero-Knowledge Proof

A commitment and a zero-knowledge proof serve different purposes.

In a commitment scheme, the committed value is eventually revealed. The commitment guarantees that the sender cannot change the value between the commitment and opening phases.

In a zero-knowledge proof, the prover may want to establish a property of a hidden value without revealing the value itself.

For example, suppose a teacher has committed to a student's mark m . Opening the commitment reveals m completely.

However, suppose the teacher wants to prove only that

$$m \geq 50$$

without revealing the exact mark.

A commitment alone does not provide this functionality. Opening the commitment would reveal the entire value.

This leads to the question:

How can we prove a property of a committed value without opening the commitment?

This is one of the fundamental motivations for zero-knowledge proofs.

6 From Commitments to Zero-Knowledge Proofs

Suppose a value m has been committed using a commitment scheme

$$C = \text{Com}(m; r).$$

We may want to prove that the committed value satisfies some property without revealing m .

For example, we may want to prove that m lies in the range $[50, 100]$. The desired proof establishes that there exists a value m and randomness r such that the commitment is valid and the value satisfies the required property.

Conceptually, the goal is

$$\text{Commit to } m \quad + \quad \text{prove a property of } m$$

without revealing m .

This idea forms the basis of many zero-knowledge protocols and is particularly useful when the committed value represents sensitive information.

7 Categories of Commitment Schemes

Commitment schemes can be classified in two different and largely independent ways.

The first classification is based on *how the commitment is constructed*. Examples include:

- Hash-based commitments.
- Discrete-logarithm-based commitments.
- RSA-based commitments.
- Lattice-based commitments.
- Code-based commitments.

The second classification is based on *what is being committed*. Examples include:

- Bit commitments.
- Message or string commitments.
- Integer or numerical commitments.
- Vector commitments.
- Polynomial commitments.
- Set commitments.

The first classification concerns the underlying cryptographic construction and security assumption. The second concerns the mathematical structure of the data being committed.

8 Commitment Construction and Security Assumptions

Different commitment constructions rely on different cryptographic assumptions.

A hash-based commitment may use a construction such as

$$C = H(m\|r).$$

A discrete-logarithm-based commitment can use a construction of the form

$$C = g^m h^r.$$

Lattice-based commitments use algebraic structures based on lattices, while other constructions may rely on RSA or coding-theoretic assumptions.

The choice of construction determines the computational properties and security assumptions of the commitment scheme.

For example, if a commitment relies on the hardness of the discrete logarithm problem, its security must be reconsidered in the presence of a sufficiently powerful quantum computer. This motivates the study of post-quantum commitment schemes based on assumptions believed to resist quantum attacks.

9 What Can Be Committed?

The committed value can have different mathematical structures.

A commitment may contain a single bit, a message, an integer, a vector, a polynomial, or a set. The commitment mechanism must be appropriately defined for the required data type and for the properties that later need to be proved.

For example, a vector can be represented as

$$\mathbf{x} = (x_1, \dots, x_n),$$

while a polynomial can be represented as

$$f(X) = \sum_{i=0}^d a_i X^i.$$

The important distinction is that the *commitment mechanism* and the *committed object* are separate concepts.

For example, a Pedersen commitment specifies a particular construction, while the value m inside the commitment could represent a bit, an integer, a field element, or another suitable algebraic object.

10 Committing to Different Types of Data

In many applications, arbitrary data can first be encoded into a suitable mathematical or binary representation.

For example, an object x can be encoded as a binary string using an encoding function

$$\text{Encode}(x) \in \{0, 1\}^*.$$

A hash-based commitment can then be constructed as

$$C = H(\text{Encode}(x) \| r),$$

where r is a random nonce.

Conceptually, the process is

$$\text{Data} \longrightarrow \text{Encoding} \longrightarrow \text{Commitment}.$$

Therefore, the main challenge is often not merely committing to the data, but enabling useful statements about the hidden data to be proved later.

11 Key Takeaways

A commitment scheme allows a party to commit to a value while keeping it hidden until a later opening.

The two fundamental security properties are:

- **Hiding**, which protects the committed value from the receiver.
- **Binding**, which prevents the sender from changing the committed value.

Hash functions provide a simple way to construct commitments using randomness. Commitments can be applied to many types of data, including bits, messages, integers, vectors, polynomials, and sets.

A commitment by itself does not allow a party to prove properties of a hidden value. This motivates zero-knowledge proofs, where a prover can establish that a committed value satisfies a particular property without revealing the value itself.